MATH 128 - Programming Assignment 2 - Jie Wen

Problem 1: Damped Newton iteration

(a)

`newton_1d_trace.m`

```
function [x, iter, p_hist, step_hist, f_hist] = newton_1d_trace(f, df, x0, tol, nmax)
% NEWTON_1D_TRACE  Newton's method with Armijo backtracking and full trace.
% Returns:
%   x         final approximation
%   iter      number of iterations performed
%   p_hist    iterates [p0, p1, ..., p_iter]
%   step_hist step sizes |p_{k+1} - p_k|
%   f_hist    residuals  |f(p_k)|

    if nargin < 5, nmax = 1000; end
    if nargin < 4, tol  = 1e-8;  end
    if nargin < 3, x0   = 1;     end
    if nargin < 2
        f  = @(x) (x.^2 - 2);
        df = @(x) (2*x);
    end

    % Line-search parameters (Armijo)
    c_armijo  = 1e-4;     % sufficient decrease constant
    tau       = 0.5;      % backtracking factor
    lambdaMin = 2^-20;    % minimum step size before giving up
    dfx_eps   = 1e-14;    % treat |df| < dfx_eps as "too small"

    % Preallocate history (trim later)
    p_hist    = zeros(nmax+1,1);
    step_hist = zeros(nmax,1);
    f_hist    = zeros(nmax+1,1);

    iter       = 0;
    p_hist(1)  = x0;
    fx         = f(x0);
    f_hist(1)  = abs(fx);
    phi        = 0.5*fx.^2;
    err        = tol + 1;

    while err > tol
        x = p_hist(iter+1);
        fx = f(x);
        dfx = df(x);

        % Newton direction p = -f/df (safe fallback if df is tiny/non-finite)
        if ~isfinite(dfx) || abs(dfx) < dfx_eps
            p = -sign(fx);              % tiny, conservative fallback step
        else
            p = -fx/dfx;
        end

        % Backtracking line search on phi(x)=0.5*f(x)^2  (Armijo)
        lambda = 1;
        accepted = false;
        phi_x = 0.5*fx.^2;

        while ~accepted
            x_new = x + lambda*p;
            fx_new = f(x_new);
            if isfinite(x_new) && isfinite(fx_new)
                phi_new = 0.5*fx_new.^2;
                % Armijo: phi(x_new) <= phi(x) - c*lambda*f(x)^2
                if phi_new <= phi_x - c_armijo*lambda*(fx.^2)
                    accepted = true;
                    break;
                end
            end

            lambda = lambda * tau;
            if lambda < lambdaMin
                % Give up backtracking; accept the best we have (to avoid stall)
                x_new  = x + lambda*p;
                fx_new = f(x_new);
                phi_new= 0.5*fx_new.^2;
```

```
                accepted = true;
            end
        end

        iter = iter + 1;
        if iter > nmax
            warning('Reached maximum iterations without meeting tolerance.');
            break;
        end

        p_hist(iter+1)  = x_new;
        step_hist(iter) = abs(x_new - x);
        f_hist(iter+1)  = abs(fx_new);

        err = step_hist(iter);   % step-size stopping
        phi = phi_new;
    end

    x = p_hist(iter+1);
    p_hist    = p_hist(1:iter+1);
    step_hist = step_hist(1:iter);
    f_hist    = f_hist(1:iter+1);
end
```

demo_newton_arctan.m

```
% DEMO_NEWTON_ARCTAN  Newton with backtracking on f(x) = atan(x)
% Two initial guesses, plot f and first 4 approximations.

clear; clc;

% f and df (MATLAB inverse tangent is atan, in radians)
f  = @(x) atan(x);          % arctan(x)
df = @(x) 1./(1 + x.^2);    % derivative of atan
% See MATLAB doc: atan(x) returns tan^{-1}(x) in radians.  :contentReference[oaicite:3]{index=3}

tol  = 1e-6;
nmax = 200;

p0_list = [1.35, 1.45];

for case_idx = 1:numel(p0_list)
    p0 = p0_list(case_idx);

    fprintf('===== Start p0 = %.4f =====\n', p0);
    try
        [x_star, iter, p_hist, step_hist, f_hist] = newton_1d_trace(f, df, p0, tol, nmax);

        % Print table
        fprintf('%5s%20s%20s%20s\n','k','p_k','|f(p_k)|','|p_{k}-p_{k-1}|');
        fprintf('%5d%20.12g%20.12g%20s\n',0,p_hist(1),f_hist(1),'   ---');
        for k = 1:iter
            fprintf('%5d%20.12g%20.12g%20.12g\n',k,p_hist(k+1),f_hist(k+1),step_hist(k));
        end
        fprintf('Converged in %d iterations to x = %.12g (|step| <= %.1e)\n\n', iter, x_star, tol);

        % Plot f and first 4 approximations
        k_show = min(4, numel(p_hist)-1);     % show p0..p3 (or as many as available)
        pts    = p_hist(1:k_show+1);          % include p0..p_kshow
        xx_min = min(pts) - 0.5;  xx_max = max(pts) + 0.5;
        if xx_min == xx_max, xx_min = xx_min - 1; xx_max = xx_max + 1; end

        xx = linspace(xx_min, xx_max, 1000);
        yy = f(xx);

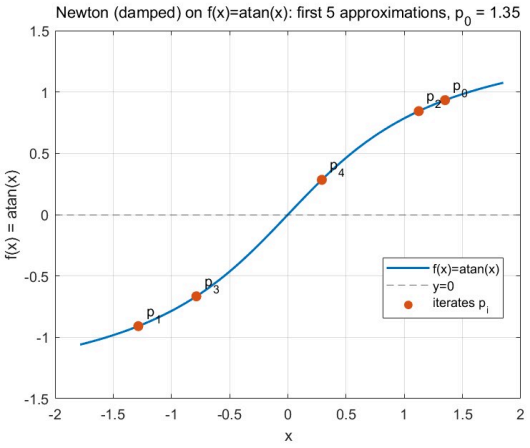
        figure('Name', sprintf('f(x)=atan(x), p0=%.2f', p0));
        plot(xx, yy, 'LineWidth', 1.5); hold on; grid on;
        yline(0,'--');
        scatter(pts, f(pts), 50, 'filled');
        for j = 1:(k_show+1)
            text(pts(j), f(pts(j)), sprintf('  p_{%d}', j-1), 'VerticalAlignment','bottom');
        end
        xlabel('x'); ylabel('f(x) = atan(x)');
        title(sprintf('Newton (damped) on f(x)=atan(x): first %d approximations, p_0 = %.2f', k_show+1, p0));
        legend({'f(x)=atan(x)','y=0','iterates p_i'}, 'Location','best');

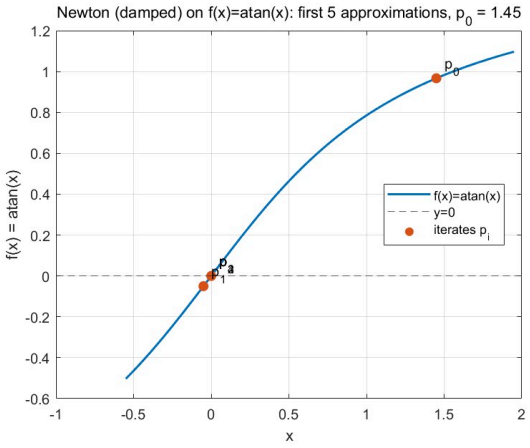
    catch ME
        % If anything still goes wrong, keep the session moving.
        warning('Run with p0=%.4f terminated: %s', p0, ME.message);
    end
end
```

Outputs:

```
===== Start p0 = 1.3500 =====
    k                 p_k            |f(p_k)|      |p_{k}-p_{k-1}|
    0                1.35      0.933247528656                 ---
    1      -1.28409114963      0.909140878958       2.63409114963
    2       1.12412310647      0.843766774872       2.40821425611
    3     -0.785871881003      0.666066670539       1.90999498748
    4      0.291553977394      0.283690251629        1.0774258584
    5    -0.0162510014433     0.0162495710669      0.307804978837
    6   2.86105489419e-06   2.86105489418e-06     0.0162538624982
    7  -1.56129348003e-17   1.56129348003e-17   2.86105489421e-06
    8                   0                   0   1.56129348003e-17
Converged in 8 iterations to x = 0 (|step| <= 1.0e-06)

===== Start p0 = 1.4500 =====
    k                 p_k            |f(p_k)|      |p_{k}-p_{k-1}|
    0                1.45      0.967046993397                 ---
    1    -0.0501316485078     0.0500897150662       1.50013164851
    2    8.3951137837e-05   8.39511376397e-05     0.0502155996456
    3  -3.94446858531e-13   3.94446858531e-13   8.39511382314e-05
    4                   0                   0   3.94446858531e-13
Converged in 4 iterations to x = 0 (|step| <= 1.0e-06)
```





(b)

`damped_newton.m`

```
function [p_hist, f_hist, rho_hist, lambda_hist, info] = damped_newton(f, df, p0, lambda_min, tol, maxit)
% DAMPED_NEWTON  Backtracking (damped) Newton method with the given monotonicity test.
%   p_{n+1} = p_n - lambda_n * f(p_n)/f'(p_n), with lambda_n chosen from
%   {1, 1/2, 1/4, ...} down to lambda_min to satisfy:
%       | f(p_{n+1}) / f'(p_n) | <= (1 - lambda_n/2) * | f(p_n) / f'(p_n) |  (★)
%
% INPUTS
%   f, df        : function handles
%   p0           : initial point
%   lambda_min   : minimum step size (e.g., 1e-3)
%   tol          : stopping tolerance on |p_n| ONLY (assignment requires |p_n| <= tol)
%   maxit        : max iterations
%
% OUTPUTS
%   p_hist       : [p0, p1, ..., pN]
%   f_hist       : |f(p_n)| at each stored iterate
%   rho_hist     : scaled residual | f(p_n) / f'(p_n) | at each iterate (same n as p_hist)
%   lambda_hist  : accepted lambda_n for each *step* (length = number of steps)
%   info         : struct with fields:
%                    .converged (logical), .it (#steps taken if converged),
%                    .stopped_reason (string)

    if nargin < 6, maxit = 10000; end
    if nargin < 5, tol = 1e-6; end
    if nargin < 4, lambda_min = 1e-3; end

    % Initialize
    p = p0;
    p_hist = p;
    f_hist = abs(f(p));
    rho_hist = [];        % we'll push rho_0 below after computing df(p0)
    lambda_hist = [];
    info = struct('converged', false, 'it', 0, 'stopped_reason', "");

    % Guard against absurd initial values
```

```
if ~isfinite(p) || abs(p) > 1e16
    info.stopped_reason = 'Invalid initial p0.';
    return
end

% The test uses df(p_n) in both LHS and RHS; compute it once per iteration
last_lambda = 1;   % doubling rule base (lambda_1 tries 1)

for n = 1:maxit

    fp  = f(p);
    dfp = df(p);

    if ~isfinite(dfp) || dfp == 0
        info.stopped_reason = sprintf('Non-finite or zero derivative at iteration %d.', n);
        return
    end

    % Scaled residual at current iterate
    s   = fp / dfp;          % Newton correction
    rho = abs(s);            % | f(p_n) / f'(p_n) |
    rho_hist(end+1) = rho;   %#ok<AGROW>

    % Stopping rule: ONLY |p_n| <= tol (as per assignment)
    if abs(p) <= tol
        info.converged = true;
        info.it = n-1;                % no move this iteration
        info.stopped_reason = 'Converged (|p_n| <= tol) before stepping.';
        return
    end

    % Choose initial trial step length
    if n == 1
        lambda_try = 1;
    else
        lambda_try = min(1, 2*last_lambda);
    end

    % Backtracking until test (★) passes or lambda < lambda_min
    accepted = false;
    while lambda_try >= lambda_min
        p_next = p - lambda_try * s;

        % Divergence protection on position (not a stop-by itself, but we avoid NaNs later)
        if ~isfinite(p_next) || abs(p_next) > 1e12
            % treat as failure of test, try smaller lambda
            lambda_try = lambda_try/2;
            continue
        end

        f_next = f(p_next);
        % LHS and RHS of the monotonicity test
        lhs = abs(f_next / dfp);
        rhs = (1 - lambda_try/2) * rho;

        if lhs <= rhs
            % Accept this step
            p = p_next;
            p_hist(end+1)  = p;             %#ok<AGROW>
            f_hist(end+1)  = abs(f(p));     %#ok<AGROW>
            lambda_hist(end+1) = lambda_try; %#ok<AGROW>
            last_lambda = lambda_try;
            accepted = true;
            break
        else
            lambda_try = lambda_try/2;
        end
    end

    if ~accepted
        info.stopped_reason = sprintf('Backtracking failed (lambda < lambda_min) at iteration %d.', n);
        return
    end

    % After taking a step, check the assignment stopping rule again
    if abs(p) <= tol
        info.converged = true;
        info.it = n;   % we just took step n
        info.stopped_reason = 'Converged (|p_n| <= tol).';
        return
    end

end
```

```
    info.stopped_reason = 'Reached maximum iterations.';
end
```

`newton_standard.m`

```
function [p_hist, f_hist, rho_hist, info] = newton_standard(f, df, p0, tol, maxit)
% NEWTON_STANDARD  Plain Newton p_{n+1} = p_n - f/df with the SAME stop rule:
%                  stop only when |p_n| <= tol (assignment requirement).
%
% Also logs |f(p_n)| and scaled residual rho_n = |f(p_n)/f'(p_n)|.
%
% Divergence detection: break if |p| > 1e12, derivative non-finite/zero,
% or max iterations exceeded. These are *not* convergence.

    if nargin < 5, maxit = 10000; end
    if nargin < 4, tol   = 1e-6;  end

    p = p0;
    p_hist = p;
    f_hist = abs(f(p));
    rho_hist = [];

    info = struct('converged', false, 'it', 0, 'stopped_reason', "");

    for n = 1:maxit
        fp  = f(p);
        dfp = df(p);

        if ~isfinite(dfp) || dfp == 0
            info.stopped_reason = sprintf('Non-finite or zero derivative at iteration %d.', n);
            return
        end

        rho = abs(fp/dfp);
        rho_hist(end+1) = rho; %#ok<AGROW>

        % Assignment stopping rule ONLY:
        if abs(p) <= tol
            info.converged = true;
            info.it = n-1;   % before stepping
            info.stopped_reason = 'Converged (|p_n| <= tol) before stepping.';
            return
        end

        % Take full Newton step
        p = p - fp/dfp;

        % Divergence guard
        if ~isfinite(p) || abs(p) > 1e12
            info.stopped_reason = sprintf('Diverged (|p|>1e12 or non-finite) at iteration %d.', n);
            return
        end

        p_hist(end+1) = p; %#ok<AGROW>
        f_hist(end+1) = abs(f(p)); %#ok<AGROW>

        if abs(p) <= tol
            info.converged = true;
            info.it = n;
            info.stopped_reason = 'Converged (|p_n| <= tol).';
            return
        end
    end

    info.stopped_reason = 'Reached maximum iterations.';
end
```

`run_tests_damped_newton.m`

```
%% RUN_TESTS_DAMPED_NEWTON
% Compares damped Newton vs. standard Newton using the assignment's
% stopping rule:  |p_n| <= 1e-6  (error to the root at 0).
% Prints iteration counts *to tolerance* when converged; otherwise prints
% why/when the method stopped (divergence or failing backtracking).

clear; clc;

lambda_min = 1e-3;
tol        = 1e-6;
maxit      = 10000;

fmt_line = repmat('-',1,70);
```

```
%% (i) f(x) = atan(x), p0 in {2, 20}
fprintf('%s\n(i) f(x) = atan(x)\n%s\n', fmt_line, fmt_line);
f  = @(x) atan(x);
df = @(x) 1./(1 + x.^2);

for p0 = [2, 20]
    [pd, fd, rhod, lambdad, infod] = damped_newton(f, df, p0, lambda_min, tol, maxit);
    [pn, fn, rhon, infon]          = newton_standard(f, df, p0, tol, maxit);

    % Steps taken are (length-1) if converged; but we report via info.it
    if infod.converged
        s_d = infod.it;
    else
        s_d = numel(pd)-1;
    end
    if infon.converged
        s_n = infon.it;
    else
        s_n = numel(pn)-1;
    end

    fprintf('p0 = %-4g | Damped: ', p0);
    if infod.converged
        fprintf('%3d iters (converged)\n', s_d);
    else
        fprintf('%3d iters (stopped: %s)\n', s_d, infod.stopped_reason);
    end

    fprintf('          Standard: ');
    if infon.converged
        fprintf('%3d iters (converged)\n', s_n);
    else
        fprintf('%3d iters (stopped: %s)\n', s_n, infon.stopped_reason);
    end
end
fprintf('\n');

%% (ii) f(x) = x^k,  k = 4, 8, 16;  p0 in {2, 50}
fprintf('%s\n(ii) f(x) = x^k, k in {4,8,16}\n%s\n', fmt_line, fmt_line);
for k = [4, 8, 16]
    f  = @(x) x.^k;
    df = @(x) k * x.^(k-1);

    for p0 = [2, 50]
        [pd, fd, rhod, lambdad, infod] = damped_newton(f, df, p0, lambda_min, tol, maxit);
        [pn, fn, rhon, infon]          = newton_standard(f, df, p0, tol, maxit);

        % Report iteration counts (to |p_n| <= tol)
        if infod.converged
            s_d = infod.it;
        else
            s_d = numel(pd)-1;
        end
        if infon.converged
            s_n = infon.it;
        else
            s_n = numel(pn)-1;
        end

        fprintf('k=%-2d, p0=%-4g | Damped: ', k, p0);
        if infod.converged
            fprintf('%3d iters', s_d);
        else
            fprintf('%3d (stopped: %s)', s_d, infod.stopped_reason);
        end

        fprintf(' | Standard: ');
        if infon.converged
            fprintf('%3d iters\n', s_n);
        else
            fprintf('%3d (stopped: %s)\n', s_n, infon.stopped_reason);
        end
    end
end
fprintf('\n');

%% (iii) f(x) = x^{1/(2k+1)}, k in {2,4,8}; p0 in {5,20}
fprintf('%s\n(iii) f(x) = x^{1/(2k+1)}, k in {2,4,8}\n%s\n', fmt_line, fmt_line);
for k = [2, 4, 8]
    alpha = 1/(2*k + 1);
    f  = @(x) x.^alpha;
    df = @(x) alpha * x.^(alpha - 1);

    for p0 = [5, 20]
```

```
        [pd, fd, rhod, lambdad, infod] = damped_newton(f, df, p0, lambda_min, tol, maxit);
        [pn, fn, rhon, infon]          = newton_standard(f, df, p0, tol, maxit);

        if infod.converged
            s_d = infod.it;
        else
            s_d = numel(pd)-1;
        end
        if infon.converged
            s_n = infon.it;
        else
            s_n = numel(pn)-1;
        end

        fprintf('k=%-2d, p0=%-4g | Damped: ', k, p0);
        if infod.converged
            fprintf('%3d iters', s_d);
        else
            fprintf('%3d (stopped: %s)', s_d, infod.stopped_reason);
        end

        fprintf(' | Standard: ');
        if infon.converged
            fprintf('%3d iters\n', s_n);
        else
            fprintf('%3d (stopped: %s)\n', s_n, infon.stopped_reason);
        end
    end
end

fprintf('\nDone.\n');
```

Outputs:

```
----------------------------------------------------------------------
(i) f(x) = atan(x)
----------------------------------------------------------------------
p0 = 2    | Damped:   5 iters (converged)
          Standard:   5 iters (stopped: Diverged (|p|>1e12 or non-finite) at iteration 6.)
p0 = 20   | Damped:   7 iters (converged)
          Standard:   3 iters (stopped: Diverged (|p|>1e12 or non-finite) at iteration 4.)

----------------------------------------------------------------------
(ii) f(x) = x^k, k in {4,8,16}
----------------------------------------------------------------------
k=4 , p0=2    | Damped:  51 iters | Standard:  51 iters
k=4 , p0=50   | Damped:  62 iters | Standard:  62 iters
k=8 , p0=2    | Damped: 109 iters | Standard: 109 iters
k=8 , p0=50   | Damped: 133 iters | Standard: 133 iters
k=16, p0=2    | Damped: 225 iters | Standard: 225 iters
k=16, p0=50   | Damped: 275 iters | Standard: 275 iters

----------------------------------------------------------------------
(iii) f(x) = x^{1/(2k+1)}, k in {2,4,8}
----------------------------------------------------------------------
k=2 , p0=5    | Damped:  12 iters | Standard:  18 (stopped: Diverged (|p|>1e12 or non-finite) at iteration 19.)
k=2 , p0=20   | Damped:  13 iters | Standard:  17 (stopped: Diverged (|p|>1e12 or non-finite) at iteration 18.)
k=4 , p0=5    | Damped:   8 iters | Standard:  12 (stopped: Diverged (|p|>1e12 or non-finite) at iteration 13.)
k=4 , p0=20   | Damped:   9 iters | Standard:  11 (stopped: Diverged (|p|>1e12 or non-finite) at iteration 12.)
k=8 , p0=5    | Damped:   6 iters | Standard:   9 (stopped: Diverged (|p|>1e12 or non-finite) at iteration 10.)
k=8 , p0=20   | Damped:   7 iters | Standard:   8 (stopped: Diverged (|p|>1e12 or non-finite) at iteration 9.)

Done.
```

(i) $f(x) = \arctan x$

start (p_0)	Damped Newton	Standard Newton	Comment
2	**5 iters (converged)**	stopped after 5 iters (diverged at 6)	Damping prevents blow-ups due to tiny $f'(x) = 1/(1+x^2)$ far from the root; backtracking guarantees sufficient decrease and global progress.
20	**7 iters (converged)**	stopped after 3 iters (diverged at 4)	Same story: full Newton takes unstable steps; damped Newton backtracks then returns to unit steps near the root.

Damped/backtracking Newton is robust and globally convergent under mild conditions; plain Newton can diverge from large $|x|$ when the derivative is very small.

(ii) $f(x) = x^k$ with $k \in \{4, 8, 16\}$

k	p_0	Damped	Standard	Comment
4	2	**51**	51	Multiple root ⇒ linear rate $(1-\frac{1}{k})$; damping quickly accepts $\lambda=1$, so counts match.
4	50	**62**	62	Same.
8	2	**109**	109	Same.
8	50	**133**	133	Same.
16	2	**225**	225	Same.
16	50	**275**	275	Same.

At roots of multiplicity k, standard Newton is only **linear** with factor $1-\frac{1}{k}$; damped Newton behaves the same once near the solution because $\lambda=1$ typically satisfies the decrease test. (No advantage expected here unless far from the root.)

(iii) $f(x)=x^{1/(2k+1)}$ with $k\in\{2,4,8\}$

k	p_0	Damped	Standard	Comment		
2	5	**12**	stopped after 18 (diverged at 19)	For x^α with $0<\alpha<1$ (nonsmooth at 0), plain Newton maps $x_{n+1}=(1-\frac{1}{\alpha})x_n$ ⇒	$1-\frac{1}{\alpha}$	>1 (divergence). Damping fixes this via step-size control.
2	20	**13**	stopped after 17 (diverged at 18)	Same.		
4	5	**8**	stopped after 12 (diverged at 13)	Same.		
4	20	**9**	stopped after 11 (diverged at 12)	Same.		
8	5	**6**	stopped after 9 (diverged at 10)	Same.		
8	20	**7**	stopped after 8 (diverged at 9)	Same.		

With fractional power $0<\alpha<1$, Newton generally **diverges** (classic counterexample $x^{1/3}$); the backtracking/damped variant **converges** in a handful of steps by enforcing sufficient decrease each iteration.

Problem 2: Fixed point, secant and Newton method

(a)

- For a sphere of radius $R=1$, the submerged part is a spherical cap of height h. Its volume is $V_{\text{cap}}=\frac{\pi h^2}{3}(3R-h)=\pi\left(h^2-\frac{h^3}{3}\right)$ (with $R=1$).
- By Archimedes' principle, at floatation the buoyant force equals the weight of the ball: $(\rho_w g)V_{\text{sub}}=(\rho g)V_{\text{sphere}}$. With $\rho_w=1$ and $V_{\text{sphere}}=\frac{4}{3}\pi R^3=\frac{4}{3}\pi$, this gives $V_{\text{sub}}=\frac{4}{3}\pi\rho$.
- Equating $V_{\text{sub}}=\pi(h^2-h^3/3)$ and cancelling π: $h^2-\frac{h^3}{3}=\frac{4}{3}\rho \iff 3h^2-h^3=4\rho \iff h^3-3h^2+4\rho=0$.

(b)

MATLAB codes:

```
function q2
    % Floating sphere depth solver (R = 1, rho_w = 1)
    % Solves h^3 - 3 h^2 + 4 rho = 0 for h in [0,2]
    % Methods: fixed-point, secant, Newton
    % Stopping: |h_k - h_{k-1}| <= 1e-5

    % Problem data
    R = 1;
    rhos = [0.0001, 0.05, 0.4, 0.6, 0.95, 0.9999];
    tol = 1e-5;
    maxit = 10000;

    fprintf('=== Floating Sphere Depth (R=1) ===\n');
    fprintf('Cubic: h^3 - 3 h^2 + 4 rho = 0\n\n');
```

```
    %% (1) Fixed-point iteration: h_{k+1} = sqrt((h_k^3 + 4 rho)/3)
    fprintf('--- Fixed-Point Iteration ---\n');
    fprintf('%8s  %12s  %8s\n', 'rho', 'h (fixed-pt)', 'iters');
    for rho = rhos
        g  = @(h) sqrt( max(0,(h.^3 + 4*rho)/3) );  % ensure radicand >= 0
        h0 = R;
        [h_star, iters] = fixed_point(g, h0, tol, maxit);
        fprintf('%8.4g  %12.7f  %8d\n', rho, h_star, iters);
    end
    fprintf('\n');

    %% (2) Secant method on f(h) = h^3 - 3 h^2 + 4 rho
    fprintf('--- Secant Method ---\n');
    fprintf('%8s  %12s  %8s\n', 'rho', 'h (secant)', 'iters');
    for rho = rhos
        f  = @(h) h.^3 - 3*h.^2 + 4*rho;
        h0 = R;
        h_1 = clamp(h0 - 1e-2, 0, 2);   % second initial guess (perturb h0)
        [h_star, iters] = secant(f, h_1, h0, tol, maxit);
        fprintf('%8.4g  %12.7f  %8d\n', rho, h_star, iters);
    end
    fprintf('\n');

    %% (3) Newton's method on same f
    fprintf('--- Newton''s Method ---\n');
    fprintf('%8s  %12s  %8s\n', 'rho', 'h (Newton)', 'iters');
    for rho = rhos
        f  = @(h) h.^3 - 3*h.^2 + 4*rho;
        fp = @(h) 3*h.^2 - 6*h;
        h0 = R;
        [h_star, iters] = newton(f, fp, h0, tol, maxit);
        fprintf('%8.4g  %12.7f  %8d\n', rho, h_star, iters);
    end
    fprintf('\n');

end

%% ---------- helpers ----------
function [x, iters] = fixed_point(g, x0, tol, maxit)
    x_prev = x0;
    x      = x0;
    iters  = 0;

    % One 'warm-up' step so that |x - x_prev| is defined
    x = clamp(g(x_prev), 0, 2);
    iters = 1;

    while iters < maxit
        x_next = clamp(g(x), 0, 2);
        if abs(x_next - x) <= tol
            x = x_next;
            iters = iters + 1;
            return;
        end
        x = x_next;
        iters = iters + 1;
    end
    % If we get here, we hit maxit; return last x
end

function [x, iters] = secant(f, x0, x1, tol, maxit)
    f0 = f(x0); f1 = f(x1);
    iters = 1; % we already have two points; next will produce x2
    while iters < maxit
        denom = (f1 - f0);
        if denom == 0
            % fall back to bisection-like small step
            x2 = clamp(0.5*(x0 + x1), 0, 2);
        else
            x2 = x1 - f1*(x1 - x0)/denom;
            x2 = clamp(x2, 0, 2);
        end
        if abs(x2 - x1) <= tol
            x = x2; return;
        end
        % shift
        x0 = x1; f0 = f1;
        x1 = x2; f1 = f(x1);
        iters = iters + 1;
    end
    x = x1;
end

function [x, iters] = newton(f, fp, x0, tol, maxit)
```

```
    x_prev = x0;
    % warm-up step
    d = fp(x_prev);
    if d == 0, d = eps; end
    x = clamp(x_prev - f(x_prev)/d, 0, 2);
    iters = 1;

    while iters < maxit
        d = fp(x);
        if d == 0, d = eps; end
        x_next = clamp(x - f(x)/d, 0, 2);
        if abs(x_next - x) <= tol
            x = x_next; iters = iters + 1; return;
        end
        x = x_next;
        iters = iters + 1;
    end
end

function y = clamp(x, a, b)
    y = min(max(x, a), b);
end
```

Outputs:

```
=== Floating Sphere Depth (R=1) ===
Cubic: h^3 - 3 h^2 + 4 rho = 0

--- Fixed-Point Iteration ---
     rho  h (fixed-pt)     iters
  0.0001     0.0115693         7
    0.05     0.2707019         8
     0.4     0.8658694        12
     0.6     1.1341252        16
    0.95     1.7292392        57
  0.9999     1.9868274       461

--- Secant Method ---
     rho    h (secant)     iters
  0.0001     0.0115693        12
    0.05     0.2707007         6
     0.4     0.8658622         3
     0.6     1.1341378         3
    0.95     1.7292993         6
  0.9999     1.9884307        12

--- Newton's Method ---
     rho    h (Newton)     iters
  0.0001     0.0115693         9
    0.05     0.2707007         5
     0.4     0.8658622         3
     0.6     1.1341378         3
    0.95     1.7292993         5
  0.9999     1.9884307         9
```

- **Fixed-point** ($h_{k+1} = \sqrt{(h_k^3 + 4\rho)/3}$) (linear in general)
  - Iterations grow sharply as $\rho \to 1$:
    $\rho = 0.0001$ (7 iters), 0.05 (8), 0.4 (12), 0.6 (16), 0.95 (57), 0.9999 (**461**).
  - This matches the theory below (factor $\approx h^*/2$, which approaches 1 as $h^* \to 2$).
- **Secant** on $f(h) = h^3 - 3h^2 + 4\rho$ (order $\approx \phi \approx 1.618$)
  - Very fast for mid-range ρ: only **3** iterations for 0.4 and 0.6.
  - Still reasonable near the extremes: 12 iters for $\rho = 0.0001$ and $\rho = 0.9999$, 6 for $\rho = 0.05$ and 0.95.
- **Newton** on the same f (quadratic when $f'(h^*) \neq 0$)
  - Also **3–5** iterations for $\rho = 0.4, 0.6, 0.05$.
  - Needs more when ρ is very small or very close to 1: **9** iterations for $\rho = 0.0001$ and 0.9999.

The numerical h values are mutually consistent across methods and lie in $[0, 2]$ as required physically (ball of unit radius).

(c)

Let $f(h) = h^3 - 3h^2 + 4\rho$, and denote the true solution by $h^*(\rho) \in (0, 2)$.

Fixed-point iteration ($h = \phi(h) = \sqrt{(h^3 + 4\rho)/3}$)

At the fixed point, $h^{*3} + 4\rho = 3h^{*2}$. Differentiate ϕ:
$\phi'(h) = \frac{1}{2}((h^3 + 4\rho)/3)^{-1/2} \cdot \frac{h^2}{1} \quad \Rightarrow \quad \phi'(h^*) = \frac{h^{*2}}{2\sqrt{(h^{*3}+4\rho)/3}} = \frac{h^{*2}}{2\sqrt{h^{*2}}} = \frac{h^*}{2}$.

- Convergence is **linear** with factor $|\phi'(h^*)| = \frac{h^*}{2}$.
- For small ρ: $h^* \ll 1 \Rightarrow |\phi'| \ll 1 \Rightarrow$ **fast linear** (7–8 iters for $\rho = 10^{-4}, 0.05$).
- As $\rho \to 1$: $h^* \to 2 \Rightarrow |\phi'| \to 1^- \Rightarrow$ **very slow** (57 and **461** iters).
  These are exactly the contraction-mapping predictions for fixed-point iteration.

Newton's method

- Newton is **quadratic** if $f'(h^*) \neq 0$ and regularity holds. Here $f'(h) = 3h(h-2)$.
  - Mid-range densities give h^* well inside $(0, 2) \Rightarrow f'(h^*)$ is comfortably nonzero $\Rightarrow$ **3–5** steps.
  - As $\rho \to 1$, $h^* \to 2$ and $f'(2) = 0$ (near-multiple root) $\Rightarrow$ **rate collapses to linear**, explaining the **9** steps for $\rho = 0.9999$.
  - For very small ρ, $h^* \approx 0$ makes $|f'(h^*)| = |3h^*(h^*-2)| \approx 6h^*$ small, so early iterations are less "Newton-like" before the quadratic regime kicks in $\Rightarrow$ **9** steps for $\rho = 10^{-4}$.

Secant method

- The classical order is $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ (superlinear).
- Like Newton, it slows when $|f'(h^*)|$ is tiny (near $\rho \approx 1$ or very small ρ), hence 12 steps at the extremes, but shines in the middle (3–6 steps).

```
Dataview (inline field '==== Start p0 = 1.3500 =====
    k                  p_k              |f(p_k)|       |p_{k}-p_{k-1}|
    0                 1.35       0.933247528656                  ---
    1       -1.28409114963       0.909140878958        2.63409114963
    2        1.12412310647       0.843766774872        2.40821425611
    3      -0.785871881003       0.666066670539        1.90999498748
    4       0.291553977394       0.283690251629         1.0774258584
    5     -0.0162510014433      0.0162495710669       0.307804978837
    6    2.86105489419e-06     2.86105489418e-06      0.0162538624982
    7   -1.56129348003e-17     1.56129348003e-17     2.86105489421e-06
    8                    0                     0     1.56129348003e-17
Converged in 8 iterations to x = 0 (|step| <= 1.0e-06)

===== Start p0 = 1.4500 =====
    k                  p_k              |f(p_k)|       |p_{k}-p_{k-1}|
    0                 1.45       0.967046993397                  ---
    1     -0.0501316485078      0.0500897150662        1.50013164851
    2     8.3951137837e-05     8.39511376397e-05      0.0502155996456
    3   -3.94446858531e-13     3.94446858531e-13     8.39511382314e-05
    4                    0                     0     3.94446858531e-13
Converged in 4 iterations to x = 0 (|step| <= 1.0e-06)'): Error:
-- PARSING FAILED -----------------------------------------------------

> 1 | ==== Start p0 = 1.3500 =====
    | ^
  2 |     k                  p_k              |f(p_k)|       |p_{k}-p_{k-1}|
  3 |     0                 1.35       0.933247528656                  ---

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```

```
Dataview (inline field '== Floating Sphere Depth (R=1) ===
Cubic: h^3 - 3 h^2 + 4 rho = 0

--- Fixed-Point Iteration ---
     rho  h (fixed-pt)     iters
  0.0001      0.0115693         7
    0.05      0.2707019         8
     0.4      0.8658694        12
     0.6      1.1341252        16
    0.95      1.7292392        57
  0.9999      1.9868274       461

--- Secant Method ---
     rho    h (secant)     iters
  0.0001      0.0115693        12
    0.05      0.2707007         6
     0.4      0.8658622         3
     0.6      1.1341378         3
    0.95      1.7292993         6
  0.9999      1.9884307        12
```

```
--- Newton's Method ---
      rho    h (Newton)     iters
   0.0001     0.0115693         9
     0.05     0.2707007         5
      0.4     0.8658622         3
      0.6     1.1341378         3
     0.95     1.7292993         5
   0.9999     1.9884307         9'): Error:
-- PARSING FAILED ------------------------------------------------

> 1 | == Floating Sphere Depth (R=1) ===
    | ^
  2 | Cubic: h^3 - 3 h^2 + 4 rho = 0
  3 |

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string, variable
```